RADIUS

Community Crime Reporting & Hotspot Mapping Application

Full Project Outline • Requirements • Feature Specification

Built with Java • JavaFX • SQL / Relational Database

Java + JavaFX	SQL Database	Map Integration	Dual Interface	Live Hotspots

1. Project Summary

Radius is a community safety application that bridges the gap between the public and law enforcement by enabling real-time, crowd-sourced crime reporting with intelligent hotspot visualisation. The platform operates across two distinct interfaces, a public-facing reporting tool and a law-enforcement dashboard, giving both communities and police a shared situational awareness of local crime patterns.

The core loop is simple: members of the public submit incident reports through the mobile/desktop app; those reports are aggregated in a central SQL database; incidents that cluster geographically are surfaced on a live map as colour-coded hotspots; and police officers review, prioritise, and action the underlying reports through a separate, secure dashboard.

2. Problem Statement & Goals

2.1 Problem Statement

Crime reporting is fragmented. Members of the public who witness low-level incidents, such as graffiti, public disturbances, suspicious behaviour – often have no convenient channel to report them and rarely receive feedback that anything was done. Police, meanwhile, lack a consolidated, real-time picture of where low-level crime is clustering before it escalates.

2.2 Project Goals

- Provide the public with a fast, frictionless channel to report incidents of any severity.
- Aggregate individual reports into meaningful, geo-spatial hotspot data.
- Give law enforcement a single, prioritised view of all community reports.
- Protect individual privacy – the public sees patterns (hotspots), never raw reports.
- Demonstrate a full-stack Java / JavaFX / SQL application with clean architecture.

3. Application Overview

Radius consists of two integrated front-ends sharing a single back-end database. Both interfaces are built in JavaFX and communicate with the SQL database via JDBC.

Component	Users	Primary Purpose
Public App	General public / community members	Submit crime reports; view hotspot map
Police Dashboard	Law enforcement officers	View all reports; prioritise and action incidents
Database Layer	Both interfaces (via JDBC)	Store, query, and serve all report and user data

Map Module	Both interfaces	Render geographical hotspots and pin locations

4. User Roles & Access Control

4.1 Public User

- Can create an account and log in.
- Can submit a new crime report (type, description, photo, location, timestamp).
- Can view the aggregated hotspot map – colour-coded by severity and frequency.
- Cannot view any individual report submitted by other users.
- Cannot access the police dashboard or any personally identifiable data.

4.2 Police / Law Enforcement User

- Separate login portal with role verification.
- Can view all individual reports in a sortable / filterable list.
- Can see the full details of each report including submitter metadata.
- Can update the status of a report (e.g. Under Review, Actioned, Closed).
- Can view the same hotspot map as the public, plus additional data layers.
- Cannot modify or delete public user accounts.

4.3 Administrator (Optional)

- Manages police officer accounts.
- Can moderate or remove fraudulent public reports.
- Access to system-wide analytics and usage statistics.

5. Feature Specification

5.1 Public Interface – Core Features

Account Management
• User registration with email and password
• Secure login / logout
• Password reset via email token
• Profile page showing submitted report history

Crime Report Submission
• Report type selector (see Section 6 for crime taxonomy)
• Free-text description field
• Optional photo / media attachment
• Location input: GPS auto-detect or manual pin drop on map
• Date and time (auto-filled, user-editable)
• Anonymous submission toggle (hides submitter identity from police view)
• Submission confirmation screen with unique report reference number

Hotspot Map (Public View)
• Interactive map centred on the user's current location
• Hotspot circles sized by frequency of reports in that area
• Colour coding: Yellow (low) → Orange (medium) → Red (high severity/frequency)
• Tap/click a hotspot to see aggregate stats (e.g. '12 incidents in the last 30 days')
• No individual report details exposed to public users
• Date range filter (last 7 days, 30 days, 90 days, all time)
• Crime type filter to isolate specific incident categories

5.2 Police Dashboard – Core Features

Secure Police Login
• Separate login flow with officer badge/ID verification
• Session management and automatic logout after inactivity
• Audit log of officer login activity

Report Management List
• Arranged list of all submitted reports
• Columns: Report ID, Type, Severity, Location, Date/Time, Status, Submitter (if not anonymous)
• Sort by any column (date, severity, status, location)
• Filter by: crime type, severity level, date range, status, suburb/region
• Priority ranking algorithm
• Quick-view panel on row click showing full report details and photo

• Status update dropdown: New → Under Review → Actioned → Closed
• Ability to add internal officer notes to a report

Police Hotspot Map (Enhanced)
• All public map features plus:
• Drill-down into a hotspot to see individual report pins
• Click a pin to open the full report detail
• Heat map overlay showing density of reports
• Toggle between public hotspot view and full pin view

6. Crime Type Taxonomy

Reports are categorised by type and mapped to a severity tier. Severity feeds directly into hotspot colour coding and police priority ranking.

Severity Tier	Examples	Map Colour
Tier 1 — Low	Graffiti, littering, noise complaints, loitering, minor vandalism	Yellow
Tier 2 — Medium	Yelling / public disturbance, property damage, theft under $500, trespassing, drug use	Orange
Tier 3 — High	Robbery, assault, domestic abuse, weapons, stalking, breaking and entering	Red
Tier 4 — Critical	Serious assault, sexual offences, armed robbery, arson	Dark Red / Pulsing

7. Priority Ranking Algorithm

The police dashboard sorts reports by a computed priority score. This score is recalculated whenever a report is loaded. A suggested formula:

```
Priority Score  =  (Severity Weight × 40)
          +  (Recency Score × 30)
          +  (Report Count in Zone × 20)
          +  (Unactioned Status Bonus × 10)
```

Factor	Detail
Severity Weight	Tier 1 = 1, Tier 2 = 2, Tier 3 = 3, Tier 4 = 4
Recency Score	Exponential decay – reports older than 7 days score lower
Report Count in Zone	Number of reports within a 500m radius in last 30 days
Unactioned Status Bonus	+1 if status is still 'New', 0 otherwise

Officers can override the algorithm order using manual sort, the computed rank is a default starting point, not a hard constraint.

8. Database Design

8.1 Core Tables

Table	Key Columns	Notes
users	user_id (PK), email, password_hash, role, created_at	Role = PUBLIC or POLICE
crime_types	type_id (PK), type_name, severity_tier, description	Reference / lookup table
reports	report_id (PK), user_id (FK), type_id (FK), description, latitude, longitude, photo_path, timestamp, is_anonymous, status	Core report record
report_status_log	log_id (PK), report_id (FK), officer_id (FK), status, notes, updated_at	Audit trail of status changes
hotspots	hotspot_id (PK), latitude, longitude, radius_m, report_count, max_severity, last_updated	Precomputed or live-aggregated
officer_notes	note_id (PK), report_id (FK), officer_id (FK), note_text, created_at	Internal police notes
sessions	session_id (PK), user_id (FK), token, expires_at	Login session management

8.2 Key SQL Operations

- SELECT reports JOIN crime_types WHERE status = 'New' ORDER BY priority_score DESC — police dashboard default view.
- SELECT reports WHERE ST_Distance(lat, lng, user_lat, user_lng) < radius — proximity query for hotspot calculation.
- UPDATE reports SET status = ? WHERE report_id = ? — officer status update.
- INSERT INTO reports (...) VALUES (...) — new public report submission.
- SELECT type_id, COUNT(*), MAX(severity_tier) FROM reports GROUP BY geo_cell — hotspot aggregation query.

9. System Architecture

9.1 Technology Stack

Layer	Technology
Language	Java (JDK 17+)
UI Framework	JavaFX (FXML + CSS styling)
Database	MySQL or PostgreSQL (or SQLite for local dev/demo)
DB Connectivity	JDBC with a lightweight DAO (Data Access Object) pattern
Map Rendering	JavaFX WebView embedding OpenStreetMap via Leaflet.js, OR MapView JavaFX library
Build Tool	Maven or Gradle (manages dependencies)
Version Control	Git / GitHub (already established)
IDE	IntelliJ IDEA

9.2 Architectural Pattern

The application should follow an MVC (Model-View-Controller) or MVVM (Model-View-ViewModel) pattern to keep UI code, business logic, and data access cleanly separated. This is especially important in JavaFX where tight coupling between FXML controllers and database calls becomes hard to maintain quickly.

Recommended Package Structure	
• com.radius.model	— Plain Java objects: Report, User, CrimeType, Hotspot
• com.radius.dao	— Database access: ReportDAO, UserDAO, HotspotDAO
• com.radius.service	— Business logic: ReportService, HotspotService, AuthService
• com.radius.controller	— JavaFX controllers bound to FXML views
• com.radius.view	— FXML files and CSS stylesheets

• com.radius.util	— Helpers: DBConnection, PasswordHasher, MapBridge
• com.radius.app	— Main entry point

10. Map Integration

The map is one of the most visually impactful parts of Radius. There are two practical approaches within a JavaFX application:

10.1 Option A — JavaFX WebView + Leaflet.js (Recommended)

Embed a WebView component in the JavaFX scene. Load a local HTML file that uses Leaflet.js (an open-source JavaScript mapping library) with OpenStreetMap tile data. Java communicates with the map via JavaScript bridge calls.

Aspect	Detail
Tile Source	OpenStreetMap (free, no API key required)
Library	Leaflet.js v1.9+ loaded from a bundled local HTML file
Java ↔ JS Bridge	WebView.getEngine().executeScript() to call JS; JSObject callbacks to Java
Hotspot Rendering	Leaflet CircleMarker with dynamic radius and fillColor based on DB data
Pin Rendering (Police)	Leaflet Marker with custom icon per crime type
Clustering	Leaflet.markercluster plugin for dense police pin view

10.2 Option B — Third-Party JavaFX Map Library

- GMapsFX (Google Maps wrapper for JavaFX) — requires Google API key, cost implications.
- JPro Maps / FXMap — commercial options, probably overkill for this project.
- Recommendation: Stick with Option A (Leaflet + OpenStreetMap) — it is free, fully controllable, and works offline for demo purposes.

10.3 Hotspot Calculation

Hotspots are calculated by dividing the map into a grid of cells (e.g. 500m × 500m) and counting the number of reports in each cell within a rolling time window. Cells exceeding threshold counts are rendered as hotspot circles. The colour is determined by the highest severity tier of any report in that cell combined with the total report count.

11. Key Screens / Views (JavaFX FXML)

Screen Name	Description
LoginView.fxml	Shared login screen with role selector (Public / Police)
RegisterView.fxml	New account creation for public users
PublicMapView.fxml	Main screen for public users — full-screen map with hotspot overlay and side panel
ReportFormView.fxml	Multi-step form for submitting a new report
ReportConfirmView.fxml	Submission confirmation with reference number
UserProfileView.fxml	View own submitted reports and account settings
PoliceLoginView.fxml	Separate login screen for officers (or same screen with role flag)
PoliceDashboardView.fxml	Split-pane: left = report list, right = map view
ReportDetailView.fxml	Full report detail panel (photo, map pin, description, status controls, notes)
PoliceFilterPanel.fxml	Sidebar filter controls (type, severity, date range, status)
AdminView.fxml (Optional)	User management and system analytics

12. Non-Functional Requirements

12.1 Security

- Passwords must be stored as bcrypt hashes — never plaintext.
- SQL queries must use PreparedStatement to prevent SQL injection.
- Police and public sessions are kept strictly separate.
- Anonymous reports must not expose the submitter's user_id in any police-facing query.

12.2 Performance

- The report list should load within 2 seconds for up to 10,000 records (use pagination — 50 rows per page).
- Hotspot recalculation can be triggered on a scheduled basis (e.g. every 5 minutes) rather than on every page load.
- Map tile loading should not block the JavaFX UI thread — use background threads with Task<Void>.

12.3 Usability

- Public interface should be usable without a tutorial — report submission in under 3 taps/clicks.
- Police dashboard must be keyboard-navigable for professional use.

- Colour-blind accessible palette — supplement colour coding with icons or text labels.

12.4 Reliability

- All database operations should be wrapped in try-catch with meaningful error messages.
- If the DB connection fails, show a user-friendly error rather than a stack trace.
- Form validation must prevent empty or malformed submissions.

13. Stretch & Bonus Features

The following are not required for a functional MVP but would elevate the application significantly if time allows:

Feature	Description
Push Notifications	Notify public users when a hotspot forms near their home address
Photo Moderation	Flag reports with uploaded photos for officer review before public hotspot inclusion
Report Voting / Verification	Allow other users to confirm (upvote) a sighting, increasing its weight in hotspot calc
Export to CSV / PDF	Police can export a filtered report list for briefings
Analytics Dashboard	Trend charts: crimes by type over time, suburb comparison, resolution rate
Multi-language Support	JavaFX resource bundles for localisation (e.g. English + one additional language)
Dark Mode	Toggle between light and dark JavaFX CSS themes
Offline Report Queue	Cache a report locally if the DB is unreachable; submit when connectivity restores
Real-time Updates	WebSocket or polling to refresh the police list and map without manual reload
Suburb / LGA Boundary Overlay	Draw official suburb boundaries on the map for jurisdictional clarity

14. Suggested Development Milestones

These milestones are suggested phases for a group project — the team should adapt them to actual sprint or deadline schedules.

Phase	Milestone	Key Deliverables
Phase 1	Foundation	DB schema finalised; JDBC connection working; User login/register functional; bare JavaFX shell with navigation

Phase 2	Core Reporting	Public report submission form; report stored in DB; basic report list in police dashboard (no map yet)
Phase 3	Map Integration	Leaflet map embedded in WebView; hotspot circles calculated and rendered; police pin view
Phase 4	Police Dashboard	Full filter/sort on report list; status updates; officer notes; priority ranking implemented
Phase 5	Polish & Testing	Input validation; error handling; CSS styling; unit tests for DAO and service layers; end-to-end test
Phase 6 (Optional)	Stretch Features	Any of the items from Section 13 that time permits

15. Testing Strategy

15.1 Unit Testing

- Test all DAO classes against a test database or in-memory H2 database.
- Test service layer logic independently of the UI (priority score calculation, hotspot aggregation).
- Use JUnit 5 as the testing framework.

15.2 Integration Testing

- Test the full flow: form submission → DB insert → hotspot recalculation → map render.
- Test police filter combinations to ensure SQL queries return correct subsets.

15.3 UI / Acceptance Testing

- Manual walkthrough of all user stories for both public and police roles.
- Test on at least two screen resolutions to ensure JavaFX layout adapts.
- Test with empty database, small dataset, and large dataset (1000+ reports).

16. Glossary

Term	Definition
Hotspot	A geographic cluster of crime reports rendered as a coloured circle on the map
Severity Tier	A 1–4 classification of crime seriousness (Low → Critical)
Priority Score	A computed numeric ranking for police report triage
DAO	Data Access Object — a Java class responsible for one table's DB operations

JDBC	Java Database Connectivity — standard Java API for SQL database access
FXML	JavaFX XML markup language used to define UI layouts
WebView	A JavaFX component that embeds a web browser for rendering HTML/JS content
Leaflet.js	An open-source JavaScript library for interactive maps
OSM	OpenStreetMap — free, open map tile data used for the map background
Anonymous Report	A report submitted with the submitter identity hidden from police view
Status	The lifecycle state of a report: New → Under Review → Actioned → Closed